

INSTITUTO DE EDUCACIÓN SUPERIOR IDAT

TECNOLOGÍA DE LA INFORMACIÓN

INFORME TÉCNICO DE PROYECTO

API RESTful para la Gestión de Solicitudes de Soporte
Técnico

CURSO: Desarrollo de los Componentes del Negocio

4to Ciclo - Sección Backend Java

Docente: Profesor del Curso de Componentes de
Negocio

Autores / Equipo:

- Integrante 1 (Líder / Backend Developer)
- Integrante 2 (Backend Developer)
- Integrante 3 (QA/ Backend Developer)

Enlace GitHub: <https://github.com/usuario/ProyectoJavaIDAT>

Fecha: Julio, 2026

LIMA - PERÚ

1. Descripción General del Proyecto

El presente proyecto backend consiste en una API RESTful desarrollada con **Java 21** y **Spring Boot 4.1.0** para el registro, consulta, actualización y eliminación (CRUD) de solicitudes de soporte técnico. Su diseño nace de la necesidad de una empresa de servicios tecnológicos que requería automatizar la atención de solicitudes de sus clientes, las cuales anteriormente se gestionaban mediante canales informales y desordenados (papel y correos electrónicos), ocasionando demoras y pérdida de información.

Para cumplir con la rúbrica académica, la aplicación gestiona la persistencia de datos directamente en **memoria del servidor**, implementando colecciones concurrentes que simulan bases de datos relacionales sin necesidad de dependencias de bases de datos físicas, asegurando un arranque instantáneo y seguro en entornos concurrentes.

2. Estructura y Arquitectura en Capas

El sistema se ha estructurado siguiendo las mejores prácticas de la industria con el patrón de **arquitectura por capas**. Esto permite un acoplamiento débil entre componentes y facilita la mantenibilidad del código:

Capa (Paquete)	Responsabilidad	Clases Clave
controller	Exponer los endpoints de la API, capturar peticiones HTTP y retornar respuestas JSON.	ClienteController, TecnicoController, SolicitudController
service	Contener la lógica del negocio e integrar las reglas y validaciones relacionales del sistema.	ClienteService, TecnicoService, SolicitudService
repository	Gestionar el acceso y manipulación de datos almacenados temporalmente en memoria (RAM).	ClienteRepository, TecnicoRepository, SolicitudRepository
model	Definir las entidades del dominio de la aplicación (objetos de datos reales).	Cliente, Tecnico, Solicitud, EstadoSolicitud, PrioridadSolicitud
dto	Objetos de transferencia para desacoplar el envío de datos	SolicitudRequest

Capa (Paquete)	Responsabilidad	Clases Clave
	externos del modelo del dominio.	
exception	Mapear y gestionar centralizadamente las excepciones de negocio y de validación de campos.	GlobalExceptionHandler, ResourceNotFoundException, ErrorResponse

3. Roles del Equipo de Desarrollo

Para emular un flujo de trabajo ágil e industrial, el equipo se dividió las responsabilidades técnicas del proyecto de la siguiente forma:

- **Integrante 1 (Diseño de Modelo y Servicios):**

Responsable de estructurar las clases del modelo de datos (`Cliente`, `Tecnico`, `Solicitud`) y sus relaciones. Diseñó e implementó las interfaces de servicio y la persistencia segura simulada en memoria mediante el uso de `ConcurrentHashMap` y `AtomicLong` en los repositorios.

- **Integrante 2 (Controladores REST y Validaciones):**

Encargado del desarrollo de la capa de controladores REST (`ClienteController`, `TecnicoController` y `SolicitudController`). Integró y configuró la validación estricta de las solicitudes utilizando la especificación de Jakarta Validation (`@Valid`, `@NotBlank`, `@Email`).

- **Integrante 3 (Manejo de Errores y Documentación):**

Implementó el interceptor global de excepciones `GlobalExceptionHandler` con la anotación `@RestControllerAdvice`. Adicionalmente, integró `Springdoc-OpenAPI/Swagger` para la autogeneración de la documentación de endpoints y lideró el plan de pruebas funcionales.

4. Explicación del Código Destacado

Manejo de Persistencia In-Memory (Simulada)

Para garantizar que las operaciones CRUD se ejecuten correctamente sin colisiones de hilos de ejecución en el servidor web, el repositorio se implementó de la siguiente forma:

```
@Repository
public class ClienteRepository {

    private final Map<Long, Cliente> clientes = new ConcurrentHashMap<>();
    private final AtomicLong idGenerator = new AtomicLong(0);

    public Cliente save(Cliente cliente) {
        if (cliente.getId() == null) {
            cliente.setId(idGenerator.incrementAndGet());
        }
        clientes.put(cliente.getId(), cliente);
        return cliente;
    }

    // findAll, findById, deleteById...
}
```

5. Informe de Pruebas Funcionales (Postman / Swagger)

A continuación se listan los casos de prueba ejecutados para garantizar el correcto funcionamiento de los endpoints CRUD, las validaciones de entrada y el manejador de excepciones.

Prueba 1: Listado General de Clientes (GET)

Endpoint: GET `http://localhost:8081/api/clientes`

Resultado esperado: Retorna la lista con los 3 clientes precargados por la semilla de datos con código `200 OK`.

[PEGAR CAPTURA DE PANTALLA POSTMAN AQUÍ - GET /api/clientes]

Prueba 2: Registro de Cliente Exitoso (POST)

Endpoint: POST `http://localhost:8081/api/clientes`

Payload de entrada: Envía nombre, email válido y teléfono.

Resultado esperado: Cliente registrado con ID incremental 4 y código `201 Created`.

[PEGAR CAPTURA DE PANTALLA POSTMAN AQUÍ - POST /api/clientes]

Prueba 3: Intento de Registro con Email Inválido (Validación Fallida)

Endpoint: POST `http://localhost:8081/api/clientes`

Payload de entrada: Email sin formato arroba ni dominio (ej: "correo_invalido").

Resultado esperado: Retorno código `400 Bad Request` con el JSON de error estructurado indicando que el correo no es válido.

[PEGAR CAPTURA DE PANTALLA POSTMAN AQUÍ - ERROR 400 EMAIL INVÁLIDO]

Prueba 4: Registro de Solicitud de Soporte (POST)

Endpoint: POST `http://localhost:8081/api/solicitudes`

Payload DTO enviado:

```
{
  "descripcion": "El servidor DNS de la oficina no resuelve dominios
externos.",
  "prioridad": "ALTA",
  "clienteId": 1,
  "tecnicoId": 1
}
```

Resultado esperado: Creación de la solicitud en estado `PENDIENTE` con el cliente y técnico completamente vinculados. Código `201 Created`.

[PEGAR CAPTURA DE PANTALLA POSTMAN AQUÍ - POST `/api/solicitudes`]

Prueba 5: Intento de Crear Solicitud con Cliente Inexistente (Excepción de Negocio)

Endpoint: POST `http://localhost:8081/api/solicitudes`

Payload DTO enviado: `"clienteId" : 999` (inexistente).

Resultado esperado: Código de error `404 Not Found` con el mensaje: *"Cliente no encontrado con ID: 999"* devuelto por el interceptor global.

[PEGAR CAPTURA DE PANTALLA POSTMAN AQUÍ - ERROR 404 CLIENTE INEXISTENTE]

6. Conclusiones y Logros del Proyecto

- **Eficiencia en el Desarrollo:** La arquitectura por capas implementada permitió al equipo avanzar de forma paralela en los controladores, lógica de negocio y repositorios sin bloqueos.
- **Seguridad y Robustez en Datos:** El uso de `Jakarta Validation` a nivel de controlador garantiza que no ingresen solicitudes incompletas o corruptas al sistema.
- **Manejo Limpio de Errores:** El interceptor global brinda una gran experiencia de desarrollo al consumir la API, ya que estandariza las respuestas de error facilitando la depuración en frontend.
- **Independencia Tecnológica:** Gracias al desacoplamiento en capas, el almacenamiento in-memory puede ser reemplazado por bases de datos físicas (JPA/SQL) en minutos sin alterar los controladores o servicios existentes.